

- [AutoRed — HPC Connection and Deployment Guide](#)
 - [Table of Contents](#)
 - [1. Architecture Overview](#)
 - [Component Summary](#)
 - [2. Prerequisites](#)
 - [On the HPC Cluster \(One-Time Setup\)](#)
 - [On Your Local Machine](#)
 - [3. Step-by-Step Deployment](#)
 - [Step 1 — Connect to OpenFortiVPN](#)
 - [Step 2 — SSH into the HPC](#)
 - [Step 3 — Navigate to the AutoRed Project](#)
 - [Step 4 — Start the Backend Server via SLURM](#)
 - [Step 5 — Check Job Status and Find the Allocated Node](#)
 - [Step 6 — Verify the Backend Started Successfully](#)
 - [Step 7 — Configure the Frontend Proxy](#)
 - [Step 8 — Start the Frontend Dev Server](#)
 - [Step 9 — Port Forward to Your Local Machine](#)
 - [Step 10 — Open the UI in Your Browser](#)
 - [Quick Reference — All Commands in Order](#)
 - [4. Frontend Setup — macOS](#)
 - [4.1 One-Time Setup](#)
 - [4.2 Daily Workflow](#)
 - [4.3 VS Code Remote SSH \(Alternative\)](#)
 - [5. Frontend Setup — Windows](#)
 - [5.1 One-Time Setup](#)
 - [5.2 Daily Workflow \(OpenSSH — Windows 10+\)](#)
 - [6.1 Results Location](#)
 - [7. Troubleshooting](#)
 - [7.1 OpenFortiVPN Won't Connect](#)
 - [7.2 SSH Connection Fails](#)
 - [7.3 SLURM Job Stuck in Pending \(PD\)](#)
 - [7.4 Backend Server Fails to Start](#)
 - [7.5 Frontend Cannot Connect to Backend](#)
 - [7.6 Port Forwarding Not Working](#)
 - [7.7 WebSocket Connection Fails](#)
 - [7.8 Quick Diagnostic Checklist](#)
 - [Appendix A: Environment Variables](#)

- [Appendix B: File Paths](#)
- [Appendix C: Port Summary](#)

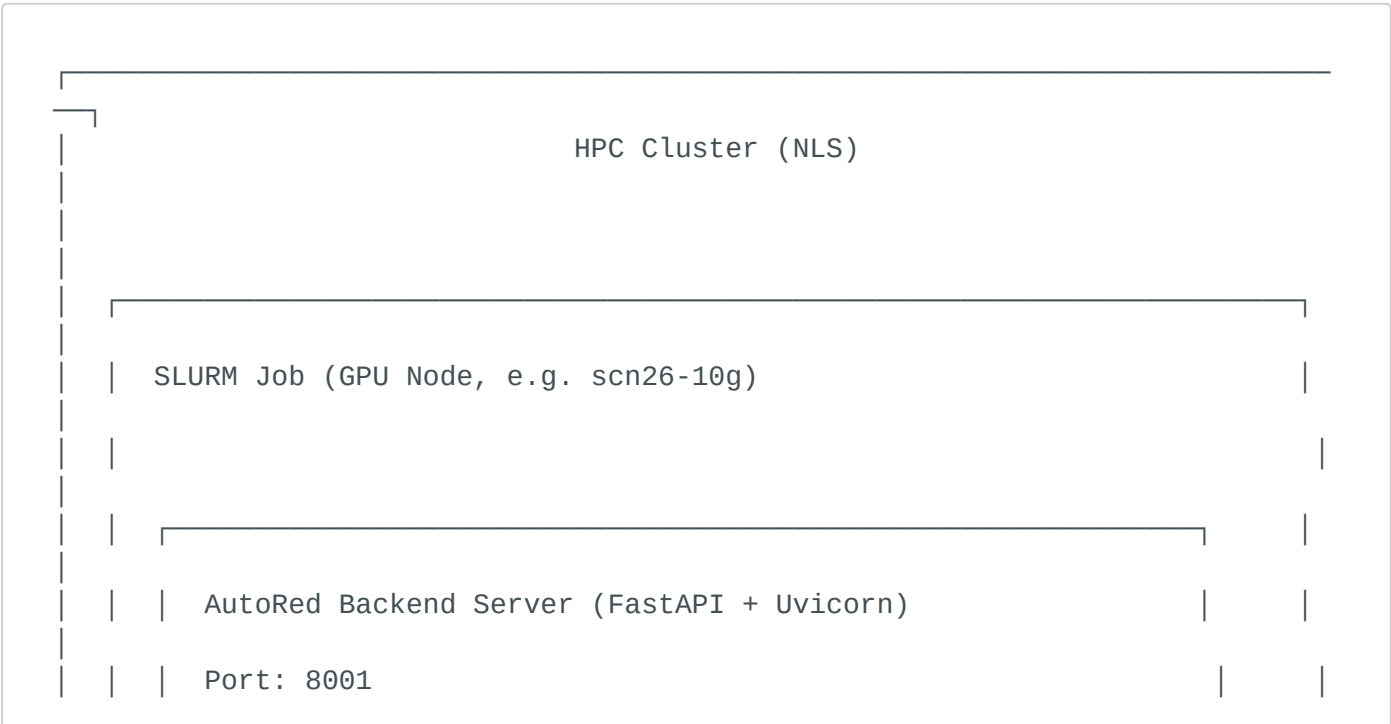
AutoRed — HPC Connection and Deployment Guide

Last Updated: 2026-06-19 HPC Gateway: 172.16.74.11 Partition: airawatp
GPU: NVIDIA A100-SXM4-40GB

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Prerequisites](#)
- 3. [Step-by-Step Deployment](#)
- 4. [Frontend Setup — macOS](#)
- 5. [Frontend Setup — Windows](#)
- 6. [Multi-GPU Benchmark Mode](#)
- 7. [Troubleshooting](#)

1. Architecture Overview





SSH Tunnel: Local:3000 ↔ HPC:3000 (frontend)

Local Mac
or Windows
:3000
(browser)

Component Summary

Component	Technology	Port	Location
Backend Server	FastAPI + Uvicorn	8001	HPC GPU node (SLURM)
Frontend Dev Server	Vite + React	3000	HPC GPU node (SSH session)
WebSocket	FastAPI WebSocket	8001	HPC GPU node (SLURM)
Models (GPU)	PyTorch + Transformers	N/A	HPC GPU node (SLURM)

2. Prerequisites

On the HPC Cluster (One-Time Setup)

- 1. Project cloned to `/nlsasfs/home/isea/isea13/AutoRed`
- 2. Virtual environment created at `.venv`
- 3. Models downloaded via `hpc/download_hf_assets.py` (offline mode)
- 4. Python dependencies installed:

```
source .venv/bin/activate
pip install -r requirements.txt
pip install -r requirements_server.txt
```

- 5. Node.js dependencies installed in `ui/`:

```
cd ui && npm install && cd ..
```

On Your Local Machine

Requirement	macOS	Windows
OpenFortiVPN client	Installed	Installed
SSH client	Built-in (<code>ssh</code>)	Built-in (Win10+) or PuTTY
VS Code (optional)	Installed	Installed

Requirement	macOS	Windows
Browser	Chrome / Firefox / Safari	Chrome / Firefox / Edge

3. Step-by-Step Deployment

This section covers the complete workflow from VPN connection to accessing the UI in your browser.

Step 1 — Connect to OpenFortiVPN

Open the OpenFortiVPN client on your local machine and connect to the Iowa State VPN.

macOS:

1. Open OpenFortiVPN from Applications
2. Select the Iowa State profile
3. Click **Connect**
4. Enter credentials when prompted

Windows:

1. Open OpenFortiVPN from Start Menu
2. Select the Iowa State profile
3. Click **Connect**
4. Enter credentials when prompted

Verify: Open a terminal and ping the gateway:

```
ping 172.16.74.11
```

You should see replies. If not, the VPN connection is not active.

Step 2 — SSH into the HPC

Open a terminal and connect to the HPC gateway:

```
ssh isea13@172.16.74.11
```

Alternative — VS Code Remote SSH:

1. Install the **Remote - SSH** extension in VS Code
2. Open the Remote SSH panel (**Ctrl+Shift+P** → **Remote-SSH: Connect to Host**)
3. Add host: **isea13@172.16.74.11**
4. Connect and open the AutoRed folder

Step 3 — Navigate to the AutoRed Project

```
cd ~/AutoRed
```

Step 4 — Start the Backend Server via SLURM

```
sbatch experiment/backendStart.sh
```

This submits a SLURM job that:

- Requests 1 GPU (A100-SXM4) on the **airawatp** partition
- Activates the **.venv** virtual environment
- Sets offline mode for HuggingFace
- Starts the FastAPI server on port **8001**

The script (**experiment/backendStart.sh**):

```
#!/bin/bash
#SBATCH --job-name=AutoRedBackend
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:A100-SXM4:1
```

```
#SBATCH --time=7-00:00:00
#SBATCH --output=logs/AutoRedBackend_%j.out
#SBATCH --error=logs/AutoRedBackend_%j.err
#SBATCH --partition=airawatp

source .venv/bin/activate

export TRANSFORMERS_OFFLINE=1
export HF_HUB_OFFLINE=1
export AUTORED_SERVER_MODE=1

python -m uvicorn server.main:app --host 0.0.0.0 --port 8001
```

Step 5 — Check Job Status and Find the Allocated Node

```
squeue --me
```

Example output:

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
1234567	airawatp	AutoRedB	isea13	R	0:30	1	scn26-10g

Key information:

- **JOBID:** **1234567** — use this to check logs
- **NODELIST:** **scn26-10g** — this is the **allocated node name** (you'll need this in Step 7)
- **ST:** **R** means Running, **PD** means Pending (wait if Pending)

Step 6 — Verify the Backend Started Successfully

Check the output log (replace **1234567** with your actual job ID):

```
# Check output log
cat logs/AutoRedBackend_1234567.out
```

```
# Or watch it in real-time
tail -f logs/AutoRedBackend_1234567.out
```

Expected output:

```
[SERVER] Starting up – loading models...
[SERVER] Loading victim LLM...
[SERVER] ✓ Victim loaded (XX.Xs)
[SERVER] Loading generator...
[SERVER] ✓ Generator loaded (XX.Xs)
[SERVER] Loading judge...
[SERVER] ✓ Judge loaded (X.Xs)
[SERVER] ✓ All models loaded in XX.Xs
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
```

If there's an error, check the error log:

```
cat logs/AutoRedBackend_1234567.err
```

Step 7 — Configure the Frontend Proxy

In the same SSH session (or a new one), navigate to the frontend directory:

```
cd ui
```

Edit `vite.config.ts` and update the proxy target to the **allocated node name** from Step 5:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    proxy: {
      '/api': 'http://scn26-10g:8001', // ← Change to your allocated node
      '/ws': {
        target: 'http://scn26-10g:8001', // ← Change to your allocated node
        ws: true,
      },
    },
  },
})
```



```
    },  
  },  
},  
})
```

Important: Replace `scn26-10g` with the actual node name from `squeue --me`. The node name changes each time you submit a new SLURM job.

Step 8 — Start the Frontend Dev Server

```
npm run dev
```

Expected output:

```
VITE v5.2.11  ready in XXX ms  
  
→ Local:    http://localhost:3000/  
→ Network:  http://scn26-10g:3000/
```

Keep this terminal open. The frontend server must stay running.

Step 9 — Port Forward to Your Local Machine

Open a **second terminal** on your local machine (keep the SSH session from Step 8 open).

```
ssh -L 3000:localhost:3000 isea13@172.16.74.11
```

This forwards the frontend dev server (running on the HPC at port 3000) to your local machine's port 3000.

Keep this terminal open. Closing it breaks the port forwarding.

Step 10 — Open the UI in Your Browser

Open your browser and navigate to:

```
http://localhost:3000/
```

You should now see the AutoRed Web UI, connected to the backend running on the HPC GPU node.

Quick Reference — All Commands in Order

```
# — Terminal 1 (SSH session – backend + frontend) —
ssh isea13@172.16.74.11
cd ~/AutoRed
sbatch experiment/backendStart.sh
queue --me # note the node name
cat logs/AutoRedBackend_<jobid>.out # verify startup
cd ui
# edit vite.config.ts → set proxy to allocated node
npm run dev

# — Terminal 2 (local machine – port forwarding) —
ssh -L 3000:localhost:3000 isea13@172.16.74.11

# — Browser —
# http://localhost:3000/
```

4. Frontend Setup — macOS

4.1 One-Time Setup

```
# Install Node.js (if not already installed)
brew install node

# Verify
node --version    # should be ≥ 18
npm --version

# Install frontend dependencies (one-time)
cd ~/AutoRed/ui
npm install
```

4.2 Daily Workflow

```
# Terminal 1: SSH into HPC
ssh isea13@172.16.74.11
cd ~/AutoRed

# Start backend
sbatch experiment/backendStart.sh
queue --me                                # note node name

# Verify backend
cat logs/AutoRedBackend_<jobid>.out

# Configure frontend proxy
cd ui
# Edit vite.config.ts → set proxy target to allocated node

# Start frontend
npm run dev

# Terminal 2: Port forward (local machine)
ssh -L 3000:localhost:3000 isea13@172.16.74.11

# Browser: http://localhost:3000/
```

4.3 VS Code Remote SSH (Alternative)

1. Install **Remote - SSH** extension
2. Add host: `isea13@172.16.74.11`
3. Connect and open `~/AutoRed`
4. Run `sbatch experiment/backendStart.sh` in the integrated terminal
5. Edit `ui/vite.config.ts` directly in VS Code
6. Run `npm run dev` in the integrated terminal

7. Port forward from a local terminal: `ssh -L 3000:localhost:3000 isea13@172.16.74.11`

5. Frontend Setup — Windows

5.1 One-Time Setup

1. **Install Node.js:** Download from <https://nodejs.org/> (LTS version)

2. **Verify:**

```
node --version    # should be ≥ 18
npm --version
```

5.2 Daily Workflow (OpenSSH — Windows 10+)

```
# Terminal 1: SSH into HPC
ssh isea13@172.16.74.11
cd ~/AutoRed

# Start backend
sbatch experiment/backendStart.sh
queue --me                                # note node name

# Verify backend
cat logs/AutoRedBackend_<jobid>.out

# Configure frontend proxy
cd ui
# Edit vite.config.ts → set proxy target to allocated node

# Start frontend
npm run dev

# Terminal 2: Port forward (local machine)
ssh -L 3000:localhost:3000 isea13@172.16.74.11

# Browser: http://localhost:3000/
```

6.1 Results Location

```
results/benchmarks/multigpu_1000r_4g/  
├─ worker_0.json  
├─ worker_1.json  
├─ worker_2.json  
├─ worker_3.json  
└─ merged_summary.json
```

7. Troubleshooting

7.1 OpenFortiVPN Won't Connect

- Ensure the VPN profile is correctly configured
- Check your credentials
- Try disconnecting and reconnecting
- Verify network connectivity: `ping 172.16.74.11`

7.2 SSH Connection Fails

```
# Test basic connectivity (after VPN is connected)  
ping 172.16.74.11  
  
# Test SSH with verbose output  
ssh -v isea13@172.16.74.11  
  
# Check SSH key permissions (macOS/Linux)  
chmod 600 ~/.ssh/id_rsa
```

7.3 SLURM Job Stuck in Pending (PD)

```
# Check queue status  
squeue --me  
  
# Check partition availability  
sinfo | grep airawatp
```

```
# Common reasons:  
# 1. No GPUs available → wait or try later  
# 2. Time limit too long → reduce --time  
# 3. Partition full → wait for jobs to complete
```

7.4 Backend Server Fails to Start

```
# Check the error log (replace with your job ID)  
cat logs/AutoRedBackend_<jobid>.err  
  
# Common issues:  
# 1. Models not downloaded → run hpc/download_hf_assets.py  
# 2. Virtual environment not activated → check .venv exists  
# 3. GPU not available → check partition and GPU type  
# 4. Port already in use → kill existing server process
```

7.5 Frontend Cannot Connect to Backend

```
# From the HPC, test direct connection to the backend node  
# Replace with your actual node name  
curl http://scn26-10g:8001/api/models/status  
  
# Expected response:  
# {"ready":true,"error":null,"victim":{"loaded":true,...},...}  
  
# If it fails, check:  
# 1. Backend server is running (check queue --me)  
# 2. Backend logs show successful startup  
# 3. vite.config.ts proxy target matches the allocated node name
```

7.6 Port Forwarding Not Working

```
# Check if SSH tunnel is active  
# On macOS:  
lsof -i :3000  
  
# On Windows:  
netstat -an | findstr 3000  
  
# Verify the SSH tunnel terminal is still open and connected
```

7.7 WebSocket Connection Fails

```
# Common causes:
# 1. vite.config.ts proxy not configured for WebSocket
#   → Ensure ws: true is set in the proxy config
# 2. Node name in vite.config.ts is incorrect
#   → Verify with squeue --me
# 3. Backend server not accepting WebSocket connections
#   → Check backend logs for WebSocket errors
```

7.8 Quick Diagnostic Checklist

Run on the HPC to diagnose issues:

```
# Check SLURM jobs
squeue --me

# Check backend logs
tail -20 logs/AutoRedBackend_*.out

# Check backend errors
tail -20 logs/AutoRedBackend_*.err

# Check if backend is listening
ss -tlnp | grep 8001

# Check if frontend is listening
ss -tlnp | grep 3000

# Test backend API directly (replace node name)
curl http://scn26-10g:8001/api/models/status
```

Appendix A: Environment Variables

Variable	Value	Purpose
AUTORED_SERVER_MODE	1	Prevents double model loading when imported by server
TRANSFORMERS_OFFLINE	1	Disable HuggingFace hub access (offline mode)

Variable	Value	Purpose
HF_HUB_OFFLINE	1	Disable HF dataset hub access

Appendix B: File Paths

Path	Description
~/AutoRed	Project root on HPC
~/AutoRed/.venv	Python virtual environment
~/AutoRed/experiment/backendStart.sh	Backend server SLURM script
~/AutoRed/ui/vite.config.ts	Frontend proxy configuration
~/AutoRed/results/	Experiment results directory
~/AutoRed/logs/	Server and worker logs

Appendix C: Port Summary

Port	Service	Location
8001	FastAPI Backend (REST + WebSocket)	HPC GPU node (SLURM)
3000	Vite Frontend Dev Server	HPC GPU node (SSH session)
3000	Port-forwarded frontend	Local machine (browser)

Document generated on 2026-06-19 Source:
/home/utsav/Github/Research/AutoRed/